Time-ordering of denoising: L2R, R2L and random

TinyGSM → GSM8K, CoBit bits model. What we tried, how, and whether it beats denoising every token at once. Report written 2 Oct 2026 from results on disk; no GPU was used.

> **Bottom line.** No ordering beat plain simultaneous denoising at any budget we tested. Ordering at decode time only (on a normally trained model) does nothing or hurts. When the model is *trained* with an order, **left-to-right is clearly the best order** (better than random, much better than right-to-left), and it learns 2–6× more per supervised token than a fair control. But the ordinary unordered model still wins at the same number of steps: **9.6% vs 8.0%** at 250k steps, and the gap was not closing.

1. The idea, in plain words

Normally CoBit denoises all 512 tokens of the answer together: at every step every token sits at the same noise level σ. An autoregressive model instead writes left to right. The hypothesis was that giving each token its own clock, so that earlier tokens become clean before later ones, would let the model condition on a settled beginning, like an AR model, and improve reasoning.

How it is implemented. Each token j gets its own noise level σ_j (all 16 bits of a token share it). A priority decides who goes first:

- **L2R**: the first answer token has the highest priority (resolves first).
- **R2L**: the last position resolves first.
- **random**: a fresh random order for every example.
- **uniform control**: no ordering, every token at the same σ (the normal model).

Two versions of the clock were used:

Design	Formula	Used in	Problem
Spread w (old)	$t_j = \text{clip}(t(1+w) - w\cdot u_j)$, $u_j \in [0,1]$ = priority normalised over the whole generated suffix	decode-only tests; w=0.25 training to 500k	The suffix is ~74% padding, so the real answer occupied a tiny slice of u: at w=0.25 93–99% of answer tokens were active at once. It never really ordered the answer.
Window k (new)	$t_j = \text{clip}(\tau(1+L/k) - r_j/k)$, r_j = absolute rank in tokens; $\sigma_j = F^{-1}(t_j)$ through one frozen noise curve shared by all arms	window study, 26 runs	Fixes the scale: about k tokens are "in flight" at a time; k=∞ is the uniform model.

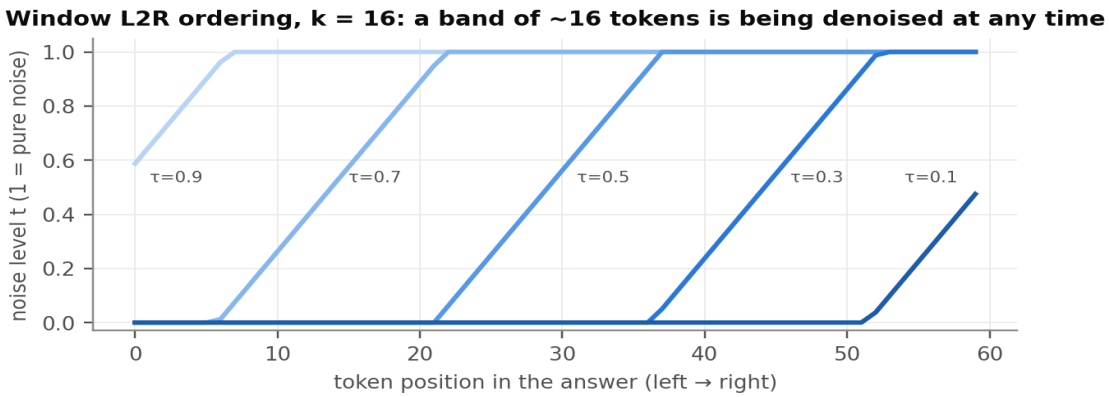


Illustration (computed from the window formula, not data): as the global time τ runs from 1 to 0, a band about k tokens wide moves from left to right; tokens to its left are already clean, tokens to its right are still noise.

In the window design, training puts loss only on tokens inside the moving band, so an ordered model sees far fewer supervised token–noise pairs per step than the uniform model (about 56 vs 462 per example at k=64, 4 at k=4). To separate *order* from *less supervision*, we added **supervision-matched controls** ("lm"): uniform σ, but each token supervised with the same probability as a window arm. Every comparison uses the same number of network evaluations (256).

2. What we ran, in order

A. Ordering only at decode time, on the production model (8 Sep)

Question: can we just decode the normally trained 500k model in L2R order? Full test set (1319 problems), 3 seeds, 256 steps, paired comparison.

Arm	Seed 0	Seed 1	Seed 2	Mean change vs control	Significant seeds
control (no order)	13.95%	14.40%	13.27%	—	—
L2R w=0.1	13.95%	13.65%	12.81%	-0.40 pp	0/3
random w=0.1	10.92%	11.14%	9.40%	-3.39 pp	3/3
L2R or random, w ≥ 0.25	0.0%			collapses to 0	(n=250 screen)

Verdict: **closed, negative.** A model trained with one shared σ does not tolerate different σ per token: a small spread does nothing, random order hurts (−3.4 pp, every seed), and a larger spread breaks generation entirely. (An early run of this test scored 0.000 everywhere because of a sampler bug that returned the second-to-last denoise; it was caught by the production anchor and fixed before these numbers.)

B. Fine-tuning the production model with ordering, w=0.25 (8–10 Sep)

Inconclusive and not used. At lr 3e-4 the runs diverged; at 3e-5 the control arm itself degenerated to 0% at 5k and 30k steps, so there was nothing to compare against. The first evaluation was also confounded: each arm decoded with its own re-fitted noise schedule.

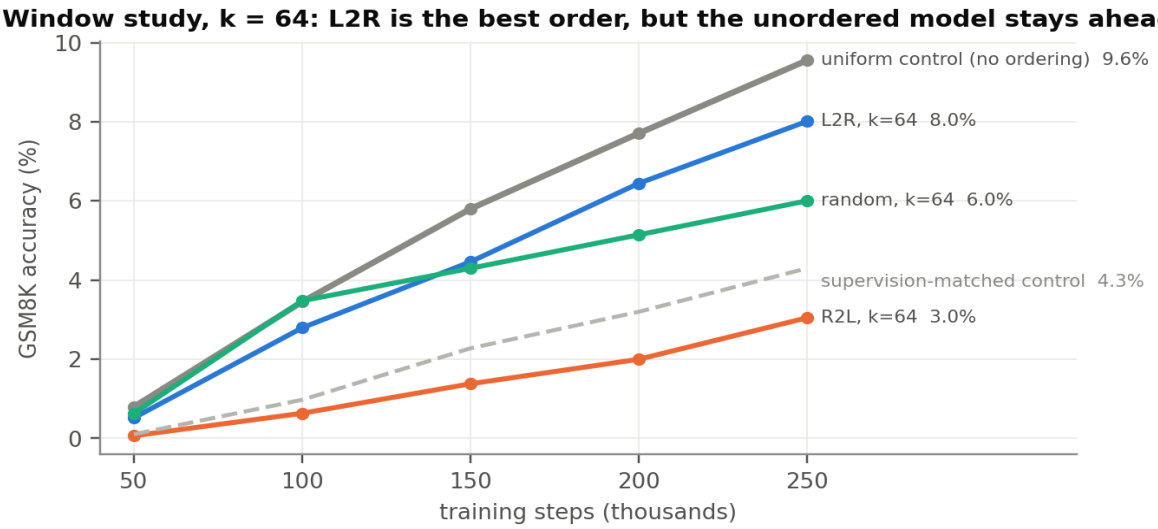
C. Training from scratch with ordering, w=0.25, to 500k steps (Sep 10–22)

Arm (1 training seed)	Decoded with its own order	Δ vs control	Decoded uniformly	Δ vs control
L2R	12.2%	-0.8 pp (0/3 sig.)	3.7%	-9.3 pp (3/3 sig.)
R2L	12.0%	-1.1 pp (0/3 sig.)	0.0%	-13.0 pp (3/3 sig.)
random	15.1%	+2.0 pp (2/3 sig.)	17.0%	+4.0 pp (3/3 sig.)
uniform control	13.0%	—	13.0%	—

At first this read as "L2R does nothing" and "random helps". The 24 Sep audit showed the design never ordered the answer: normalising the priority over a suffix that is ~74% padding gave the whole answer only a ~2× σ range (adjacent tokens differed by 0.6%). L2R and R2L were effectively simultaneous denoising with a global noise shift; only random actually perturbed the answer, and its arm also trained at systematically lower noise. **These results are not evidence about ordering in either direction**, and they rest on one training seed. The one solid fact: ordering-trained models are captive to their decode schedule (L2R decoded uniformly falls to ~4%, R2L to 0%).

D. The window study: the real test (24–26 Sep, Isambard)

26 runs from scratch: uniform control; L2R, R2L, random at k = 64, 16, 4; supervision-matched controls at the same k; each with training seeds 42 and 43. lr 1e-4, batch 512, 4 GH200 each. Stopped by budget at 250k evaluated steps (~2,780 GPU-h). Each model is decoded with its own window sampler, NFE 256, full 1319 problems.



GSM8K accuracy (mean of 2 training seeds) vs training steps, k = 64. Source: results/ordering/window_summary.json.

Arm at 250k	k = 64	k = 16	k = 4
L2R	8.0%	6.5%	1.2%
random	6.0%	3.2%	0.0%
R2L	3.0%	2.7%	1.0%
supervision-matched control	4.3%	1.1%	0.1%
uniform control (k = ∞)	**9.6%**		

Contrast at 250k (pp, Welch 95% over 2 training seeds)	k = 64	k = 16
L2R − R2L	+5.0 [-3.7, +13.6]	**+3.8 [+0.3, +7.3]**
L2R − random	+2.0 [-7.6, +11.6]	+3.2 [-1.0, +7.5]
L2R − matched control	+3.7 [-1.8, +9.2]	**+5.3 [+0.1, +10.6]**
L2R − uniform control	**-1.5 [-2.0, -1.1]**	-3.1 [-9.6, +3.5]

Bold = interval excludes 0. With only 2 training seeds these intervals have about 1–2 degrees of freedom, so most are wide; at 200k L2R also beat random at k=16 and the matched control at both k.

3. What it means

What worked

- **Order matters, in the autoregressive direction.** Among ordered models L2R is best at every k where anything is learned (L2R > random > R2L). This is the first evidence in the project that the *order itself*, not just noise heterogeneity, changes what the model learns.
- **L2R learns more per supervised token.** Against the supervision-matched control it is ~2× better at k=64 (8.0% vs 4.3%) and ~6× at k=16 (6.5% vs 1.1%).

What did not

- **No ordering beats the uniform model at equal training steps.** L2R k=64 trails by −1.5 pp [−2.0, −1.1] at 250k, and the gap is not shrinking (−1.3 at 200k).
- **Narrow windows fail:** k=4 is ~1% in every arm, including its matched control.
- **Decode-time ordering of a normal model** does nothing (small spread) or hurts (random, larger spread).
- **Ordered models only work with their own sampler:** window-trained L2R/R2L score 0% with the standard decoder.

- **Random order is never the best order**, so the "mixed-noise helps" idea from the w=0.25 study is not supported by the proper test.

Why (the explanations on record)

- **Supervision volume.** An ordered model supervises 8–116× fewer token–noise pairs per step; at this budget the uniform model's sheer volume of training signal wins.
- **Too few denoising steps per token at decode.** With 256 evaluations a token spends only ~31 steps inside a k=64 window and ~2 inside k=4. The NFE-1024 cells that would test this were never run.
- **Schedule captivity.** A model trained under one per-token noise pattern breaks under another, which is also why decode-only ordering failed.

4. How much to trust this

- **Only 2 training seeds** in the window study, no correction for the many contrasts (3 k × 4 contrasts × 5 checkpoints). The w=0.25 study had 1 seed.
- **Training bugs found afterwards** (fixed 1 Oct, not yet re-run): GPUs other than rank 0 trained with σ_{data} 0.5 instead of the ≈0.40 estimate (T1); all GPUs drew the same noise (T3); seeds 42 and 43 saw the same data order (T4), so the seed intervals understate variance. These affect all arms alike, but the conclusions are not confirmed on the fixed trainer.
- **Random-order evaluations reused the same permutations in every batch** (T5), so "random" was less random than intended.
- **Missing cells:** no 300k and no NFE-1024 evaluations, although the 300k checkpoints exist for all 26 runs.
- R2L and random rank the whole canvas including padding (the answer length is unknown at sampling), so R2L resolves the padding first; that partly explains why it is worst.

5. What to keep and what to do next

> **Keep:** uniform simultaneous denoising as the default. **Keep the window code** (committed 1 Oct with its frozen noise table): it is the correct way to test ordering. **Keep the finding** that L2R is the right direction and is more data-efficient per supervised token. **Drop:** decode-time ordering of uniform models, the w-spread design, and R2L/random as candidates.

If ordering is revisited (GPU estimates; nothing has been run):

- First confirm the training fixes (12k-step test, ~20 GPU-h for 3 seeds).
- Evaluate the existing 300k checkpoints and NFE-1024 for L2R k=16/64 (~65 GPU-h): does giving each token more decode steps close the gap?
- Test a supervision rule that also trains tokens not yet in the window (the AR-like signal), so ordered models are not starved of supervision.
- More training seeds (with T4 fixed) before claiming any small difference.

Sources. docs/experiments_ordering_objective.md (A1–A3, 20/24/26 Sep sections), docs/report_since_friday/report.tex, docs/report_meeting/report.tex, CODEBASE.md (T1–T5, R5), docs/cpu_reanalysis.md; results/ordering/confirm_summary.json, summary.json, train_curve_gh200.json, train_summary_500k_gh200.json, window_summary.json, window_sigma_field.json. Welch intervals recomputed from the per-seed accuracies in window_summary.json with the formula of experiments/ordering/analyze_window.py.